SC3020 Database Systems Principle
Group 25 Project 2 Report

Name	Contribution
Ong Hong Xun	20%
Clarence Tan Yan Kai	20%
Wong Rong Jing	20%
Chen ZhongJiang	20%
Chan Zi Jian	20%

College of Computing and Data Science
Nanyang Technological University, Singapore

2025/26 Semester 2

Source: https://github.com/HXong/SC3020_Group25/tree/project2-main/sc3020-project2

Table Of Content

1. Architecture Overview

1.1 Project.py

The entry point of the project. This file routes the query requests to other modules and launches the GUI.

1.2 Preprocessing.py

Uses psycopg2 library to connect to PostgreSQL database. Obtains Query Execution Plan (QEP) by appending EXPLAIN (FORMAT JSON) to the start of each SQL query input and generates Alternative Query Plans (AQP) to compare costs of alternative plans.

1.3 Annotation.py

Parses the raw SQL query, QEP and AQP to infer annotations to be mapped back to the SQL query, which will be sent to the interface for displaying.

1.4 Interface.py

Uses tkinter library to render the GUI for inputting SQL query, selecting example queries and displaying annotations. Uses networkx and matplotlib libraries to display the QEP as a directed graph.

2. Key Algorithms

2.1 Preprocessing.py

2.1.1 AQP Generation

For each query, 5 AQPs are generated, each disabling a single type of join/scan as follows to see the effect. The AQPs are sent to be used by the annotation algorithm.
"SET enable_seqscan = off;"
"SET enable_indexscan = off;"
"SET enable_hashjoin = off;"
"SET enable_mergejoin = off;"
"SET enable_nestloop = off;"

2.2 Annotation.py

2.2.1 SQL Parsing

A SQL query is first broken down into its constituent keywords such as FROM, WHERE.
In extract_sql_components(query), each keyword is found using regular expressions and used to slice the query into its clauses. As our algorithm uses clause-level annotations, this will be used later to map annotations to clauses.

Example SQL:
```
SELECT o.o_orderkey, l.l_linenumber, l.l_quantity
FROM orders o
JOIN lineitem l ON o.o_orderkey = l.l_orderkey
ORDER BY o.o_orderkey;
```

Extracted SQL:
```
select_clause: SELECT o.o_orderkey, l.l_linenumber, l.l_quantity
from_items:
  - table: orders
    alias: o
  - table: lineitem
    alias: l
join_conditions:
  - raw: o.o_orderkey = l.l_orderkey
order_by_conditions:
  - raw: o.o_orderkey
```

2.2 QEP Parsing

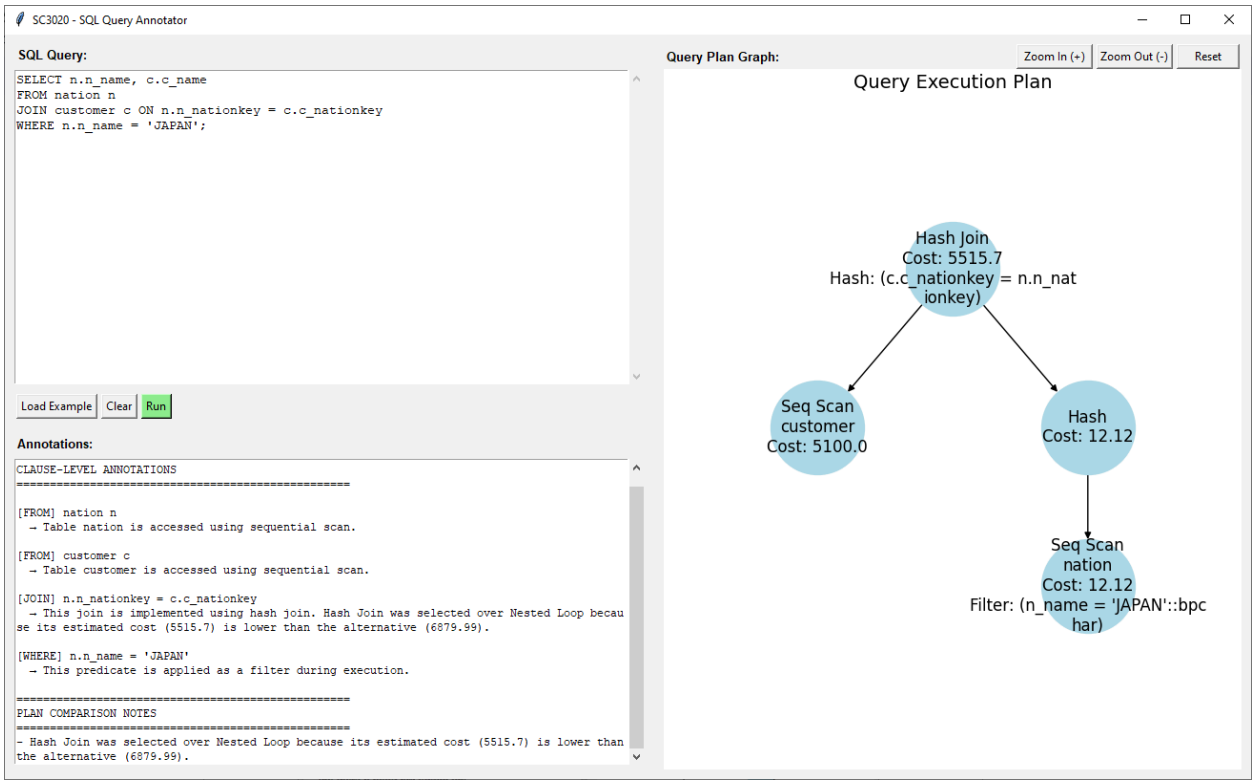
The Query Execution Plan (QEP) is obtained from PostgreSQL and traversed recursively using Depth-First Search using traverse_plan(), adding details of each node such as Child Nodes, Costs and Join Type and saving them into a dictionary. Each node is then parsed to create annotations to describe their use.

2.2.1 Join Annotation

When alternative query plans are generated, we include annotation to show that the alternative joins were estimated to have a higher cost and thus were not selected.

Example SQL:
SELECT n.n_name, c.c_name
FROM nation n
JOIN customer c ON n.n_nationkey = c.c_nationkey
WHERE n.n_name = 'JAPAN';

Relevant Annotation:
[JOIN] n.n_nationkey = c.c_nationkey
→ This join is implemented using hash join. Hash Join was selected over Nested Loop because its estimated cost (5515.7) is lower than the alternative (6879.99).



2.2.2 Sort Annotation

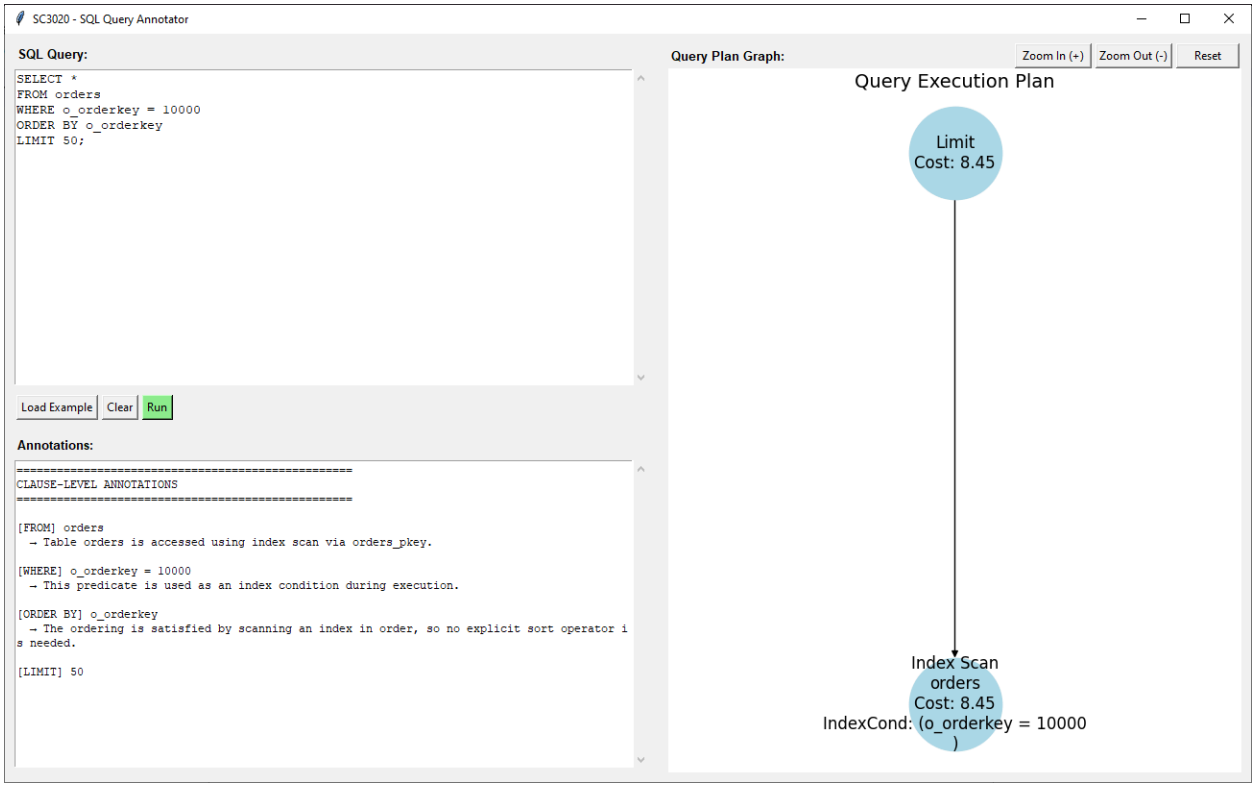
When an “ORDER BY” keyword exists in the extracted SQL but Sort operator does not exist in the QEP, we check if “is_order_satisfied_by_index” and annotate to explain why sort operation was omitted.

Example SQL:

```
SELECT *
FROM orders
WHERE o_orderkey = 10000
ORDER BY o_orderkey
LIMIT 50;
```

Annotation:

[ORDER BY] o_orderkey
→ The ordering is satisfied by scanning an index in order, so no explicit sort operator is needed.



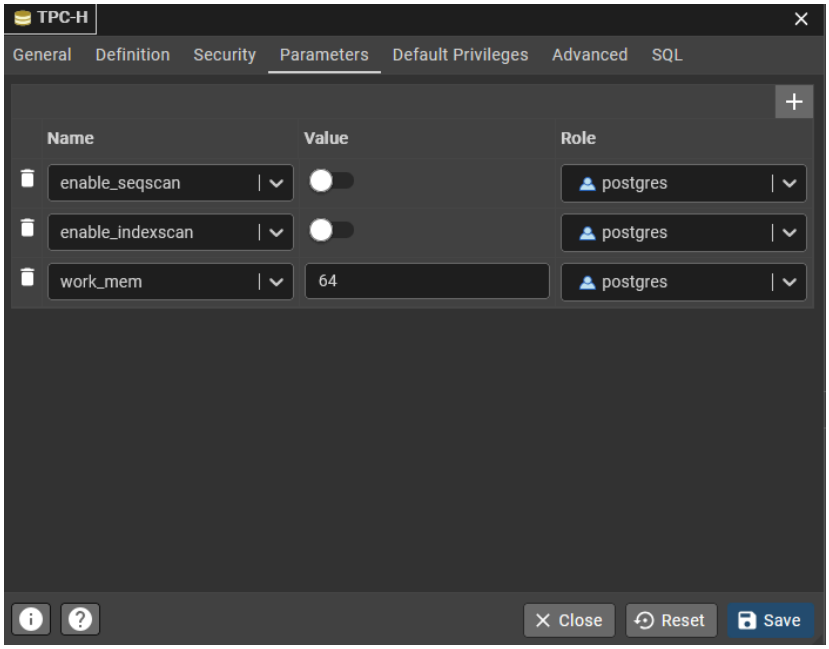
2.2.2 Recheck Annotation

When performing Bitmap Index Scan, if the bitmap gets too large for working memory, it swaps to lossy mode, which only remembers which pages contain at least 1 matching tuple instead of storing every tuple individually. This forces the DBMS to return to each flagged page to recheck

the scan condition for which exact tuples to actually return. When “recheck_cond” is present, we annotate that "This predicate is rechecked after index-based access."

Setup:

```
-- Add parameters in pgAdmin/psql to force bitmap scan and recheck for only this query
SET work_mem = '64kB';
SET enable_indexscan = off;
SET enable_seqscan = off;
```

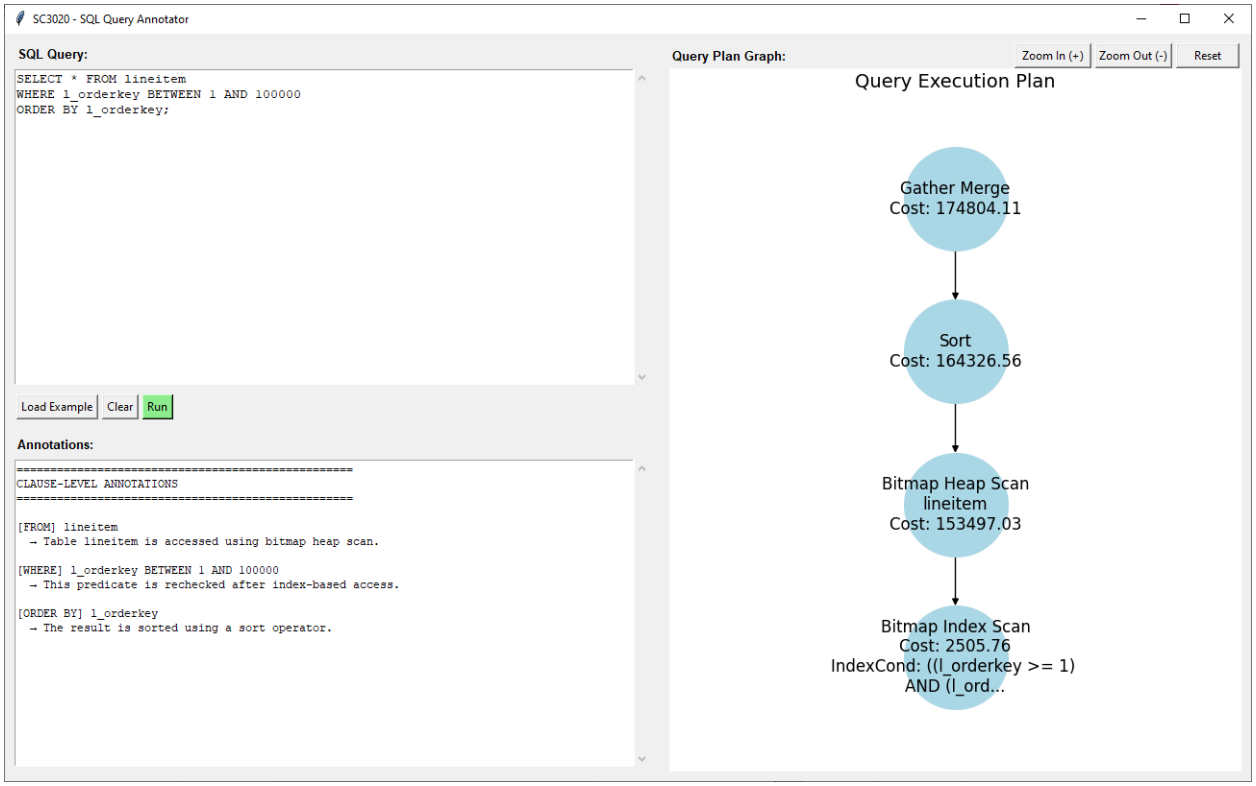


SQL Query:

```
SELECT * FROM lineitem
WHERE l_orderkey BETWEEN 1 AND 100000
ORDER BY l_orderkey;
```

Relevant Annotation:

[WHERE] l_orderkey BETWEEN 1 AND 100000
→ This predicate is rechecked after index-based access.



2.3 Design Rationale and Generality

The system adopts a rule-based approach to map SQL clauses to Query Execution Plan (QEP) operators. Instead of relying on exact string matching, conditions are first normalized into canonical forms like equality, range and pattern-matching predicates). This allows the system to match semantically equivalent expressions between SQL queries and execution plan conditions, even when they differ syntactically.

To improve generality, the annotation logic is designed to operate independently of specific queries or schemas. The SQL parsing stage extracts clause-level components (FROM, WHERE, JOIN, GROUP BY, ORDER BY), while the QEP is traversed recursively using a depth-first search to collect operator information.These two representations are then linked through normalized condition matching, enabling the system to generate annotations for a wide range of queries, including joins, filters, aggregation, and sorting.

Additionally, alternative query plans (AQPs) are generated by selectively disabling planner options. This allows the system to provide comparative explanations (e.g., why a hash join is chosen over a nested loop), improving interpretability beyond basic operator identification. Overall, this design prioritizes extensibility and adaptability, allowing the system to handle unseen queries without hardcoding specific cases.

3. User Interface

3.1 Layout

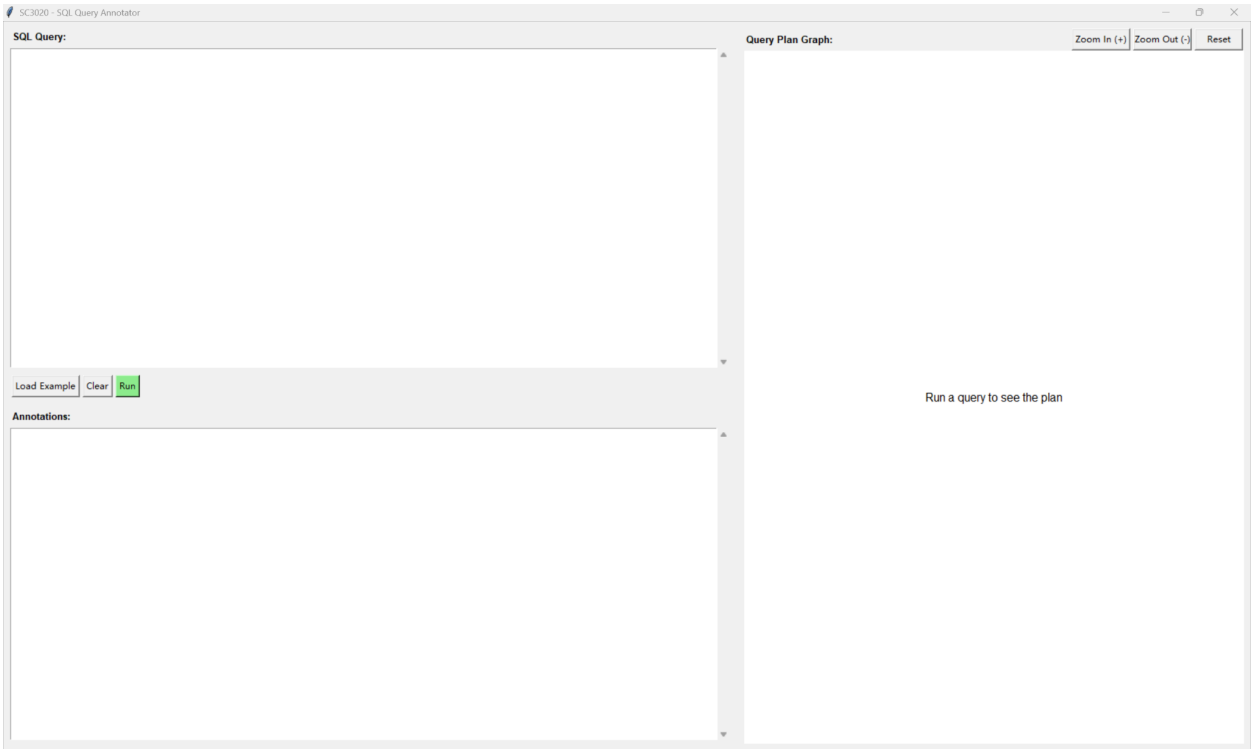
The GUI is built using Python's tkinter library. It uses a simple two-panel layout:

Left Panel:

- Contains the SQL query input area, control buttons, and annotation output area.

Right Panel:

- Displays the Query Execution Plan (QEP) as an interactive directed graph



3.2 Components

Components	Function
SQL Query Input	Multi-line text area for typing or pasting SQL queries
Load Example Button	Opens a pop-up window with 7 pre-defined example queries
Clear Button	Clears query input, annotation output, and query plan graph.
Run Button	Executes query and generates annotations

Zoom In (+) Button	Zooms into the graph for detailed view
Zoom Out (-) Button	Zooms out to see more of the graph
Reset Button	Resets the graph to its original zoom level and position
Annotations Box	Displays clause-level execution details in a categorized text format

3.3 Interactive Graph Features

The QEP graph supports the following interactive features:

Feature	How to Use
Pan (Move)	Click and hold left mouse button to drag the graph
Zoom In	Mouse scroll wheel up OR click Zoom In (+) button
Zoom Out	Mouse scroll wheel down OR click Zoom Out (-) button
Reset View	Click Reset button

3.4 Example Selector Pop-up

When the user clicks "Load Example", a modal pop-up window appears with 7 example queries:

No.	Example Name	Description
1	Simple Join	Basic two-table equi-join with LIMIT
2	Index Scan (Point Query)	Single row lookup by primary key
3	Filter with Condition	Range scan on non-indexed column
4	Order By	Sorting without an index
5	Three-way Join	Joining three tables (customer → orders → lineitem)
6	Group By	Aggregation with GROUP BY
7	Between Condition	Range query on the date column

3.5 Annotation Display Format

After running a query, the annotations box shows categorized execution details:

Annotations:

```
================================================
CLAUSE-LEVEL ANNOTATIONS
================================================

[FROM] nation n
  → Table nation is accessed using sequential scan.

[FROM] customer c
  → Table customer is accessed using sequential scan.

[JOIN] n.n_nationkey = c.c_nationkey
  → This join is implemented using hash join. Hash Join was selected over Nested Loop becau
se its estimated cost (5515.7) is lower than the alternative (6879.99).

[WHERE] n.n_name = 'JAPAN'
  → This predicate is applied as a filter during execution.

================================================
PLAN COMPARISON NOTES
================================================
- Hash Join was selected over Nested Loop because its estimated cost (5515.7) is lower than
the alternative (6879.99).
```

3.6 Graph Visualization

The QEP is converted into a directed graph using the NetworkX library. Each node represents an operator and displays:

- Operator type (Seq Scan, Hash Join, Index Scan, etc.)
- Table name (if applicable)
- Estimated cost
- Additional operator details (Hash condition, filter condition, etc.)

The graph uses a hierarchical layout algorithm to position nodes in a tree-like structure, making the execution flow easy to understand. The graph is drawn using Matplotlib and embedded into the tkinter window using FigureCanvasTkAgg.

3.7 Workflow Example

Step	Action
1.	User clicks "Load Example" and selects "Simple Join".
2.	The query appears in the SQL Query box.
3.	User clicks "Run".
4.	System connects to PostgreSQL and retrieves the execution plan.
5.	Right panel displays the interactive plan graph
6.	Left panel bottom shows clause-level annotations
7.	User can zoom in/out using mouse wheel or buttons
8.	User can pan by dragging the graph
9.	User can click "Clear" to start over

4. Limitations

Our implementation is functioning as expected. However, it also comes with several limitations. Firstly, SQL parsing is regex-based (not a full SQL parser), which only works on certain syntax and will fail on others such as subqueries (IN (SELECT ...)) and complex predicates may not be annotated. The system uses heuristic-based matching (e.g., index naming conventions and predicate normalization), which may not fully capture all complex SQL constructs.

Secondly, our graph uses Networkx's spring layout algorithm. In large plans with numerous nodes, the visuals may appear cluttered with labels and descriptions overlapping each other and cut off in cases of long description. This makes it difficult for the query writer to interpret the query plan. Therefore our implementation is more effective on smaller plans.

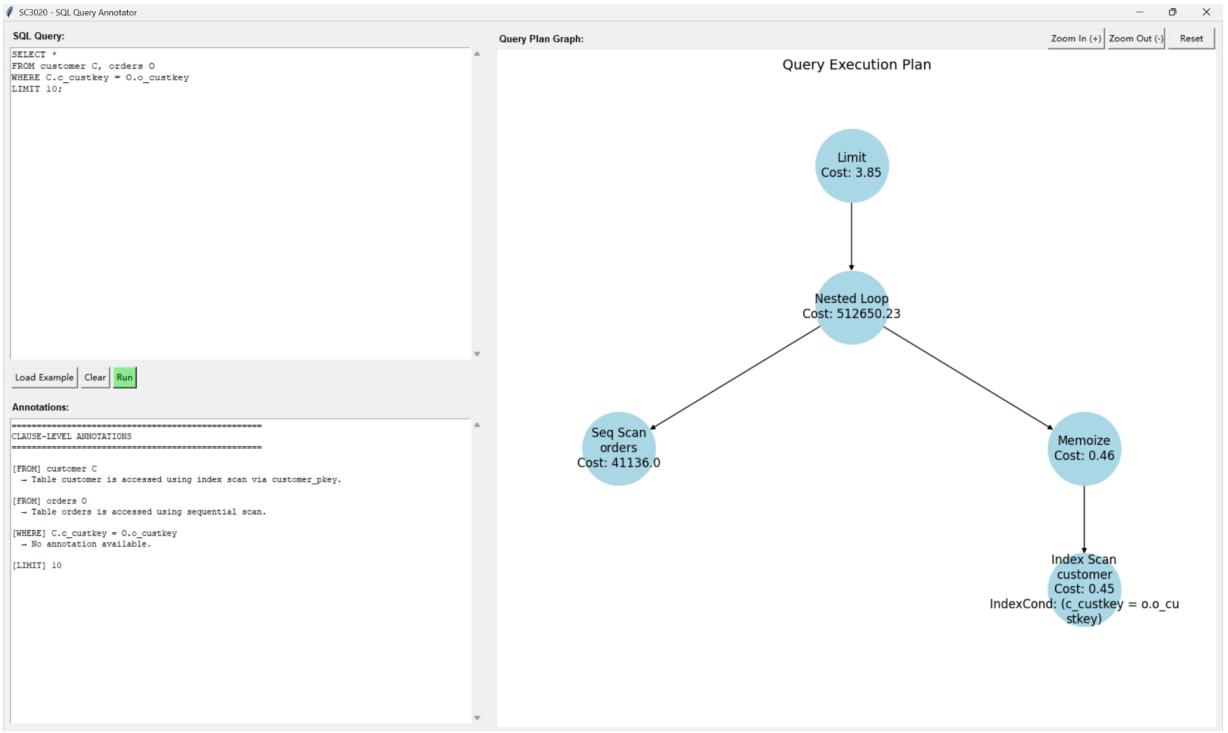
Lastly, the use of SET is not supported in the SQL input as we append EXPLAIN (FORMAT JSON) to the beginning of each query.

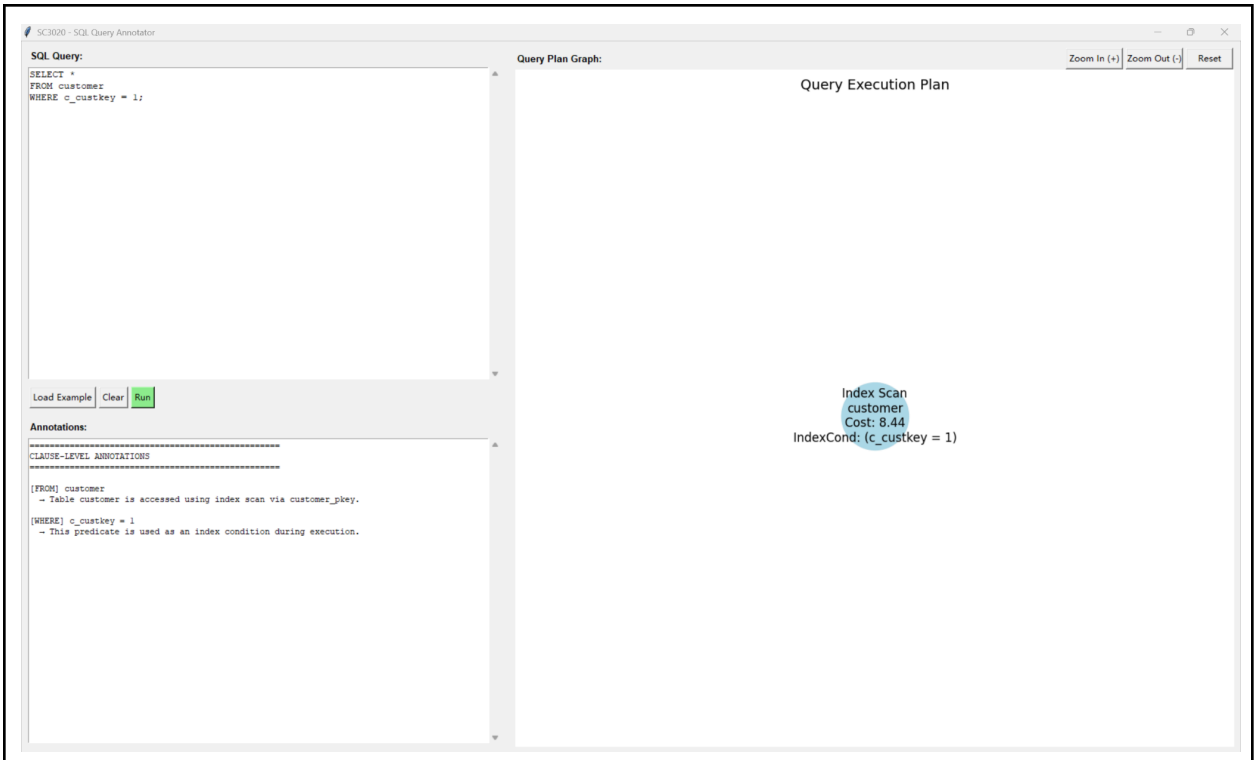
5. Conclusion

In conclusion, our application combines QEP extraction, Alternative Query Plan comparison, and clause-level annotation with an interactive tkinter-based GUI to observe both the query execution process and the estimated cost of the plans. With automatic annotation and graph visualization, our application helps users better understand how different components of a query are executed in PostgreSQL and why specific operators are selected over the others.

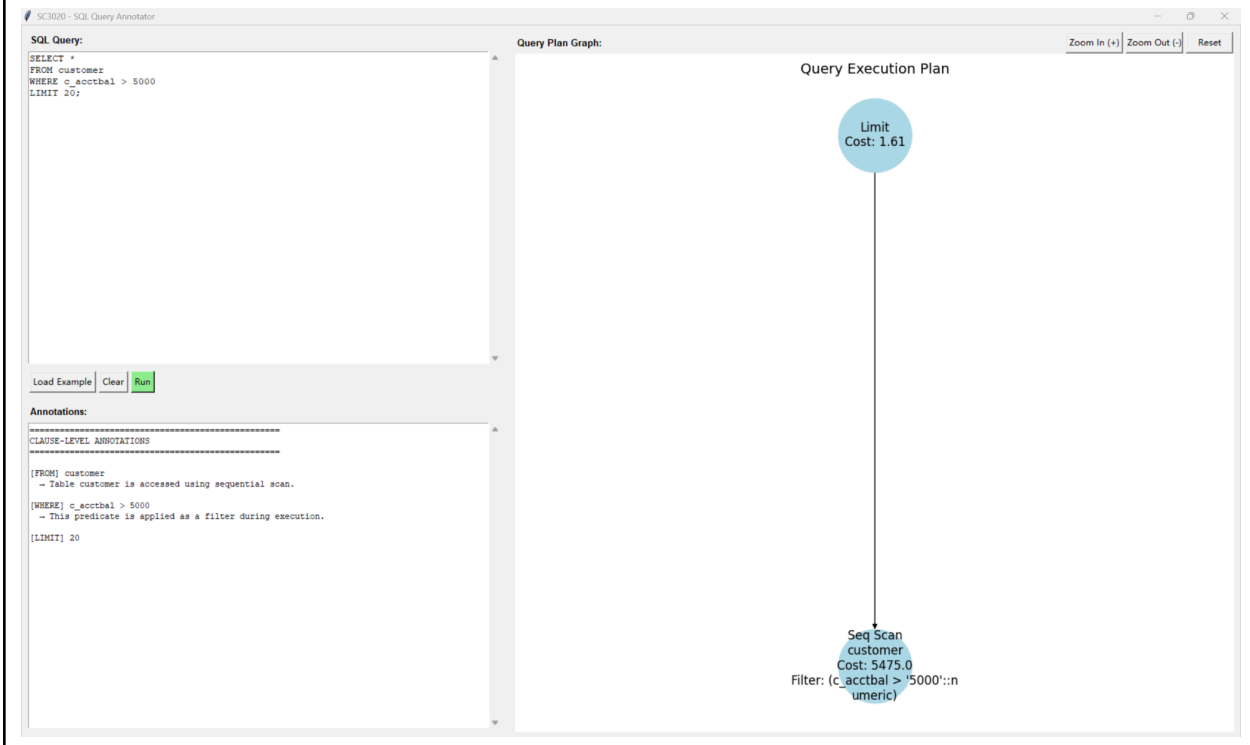
Appendix

Sample Output of Example Queries:

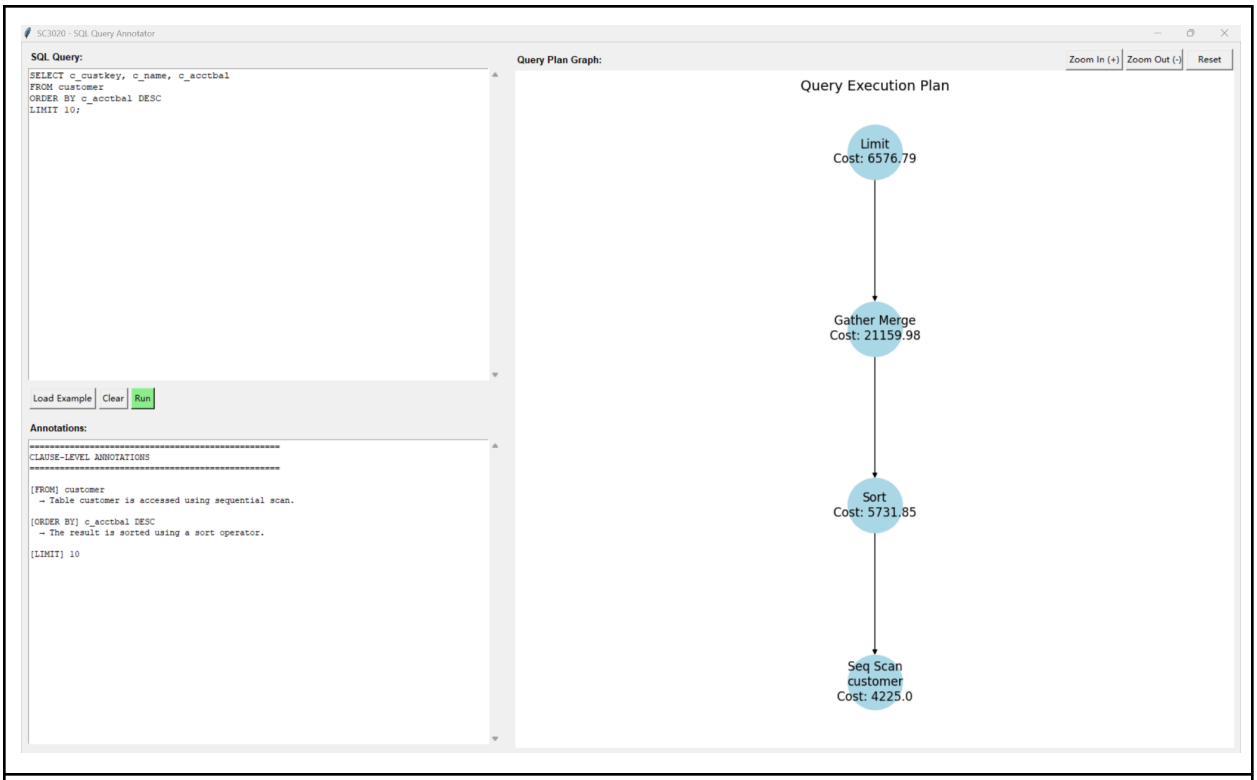
Simple Join

Index Scan (Point Query)



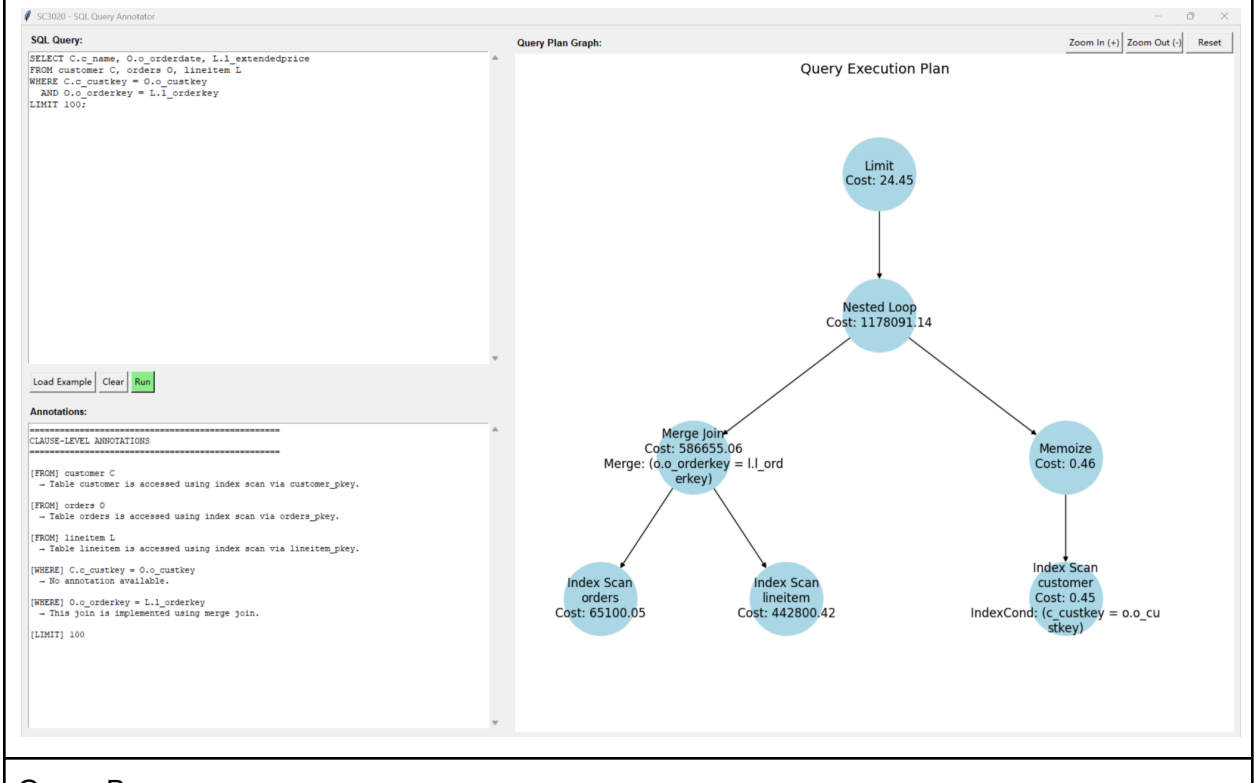
Filter with Condition



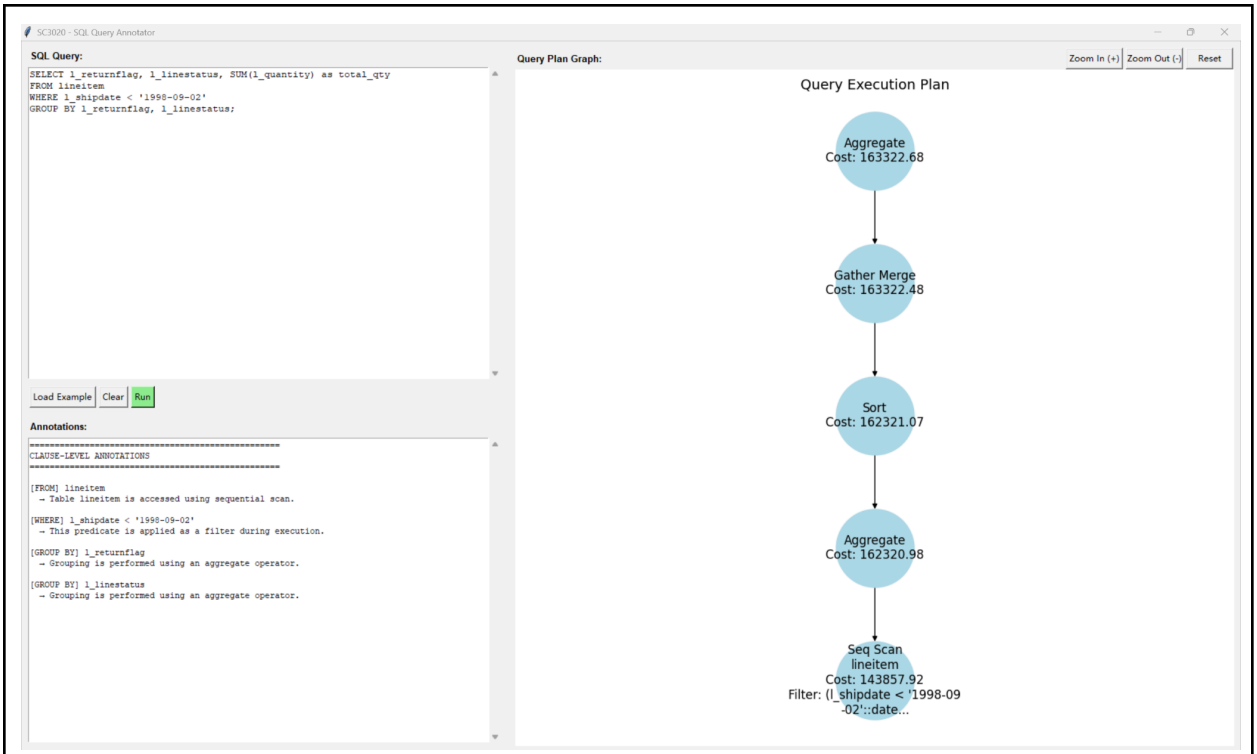
Order By



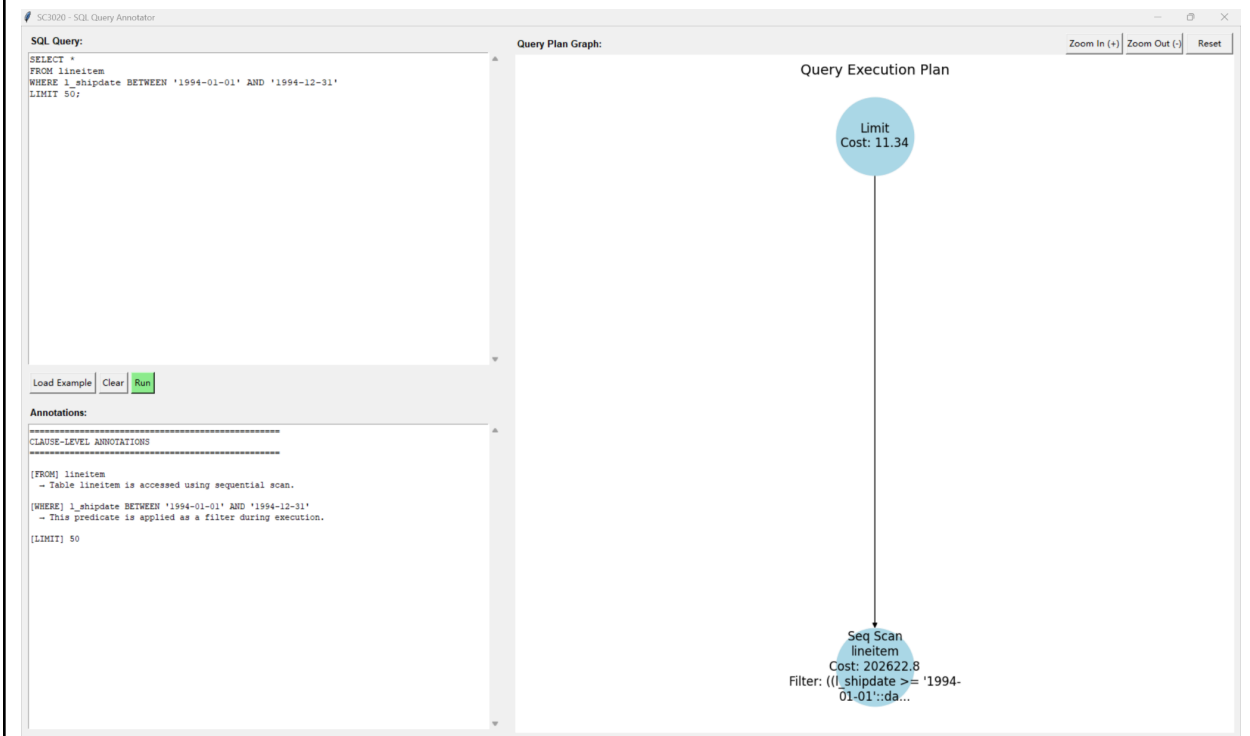
Three-way Join



Group By

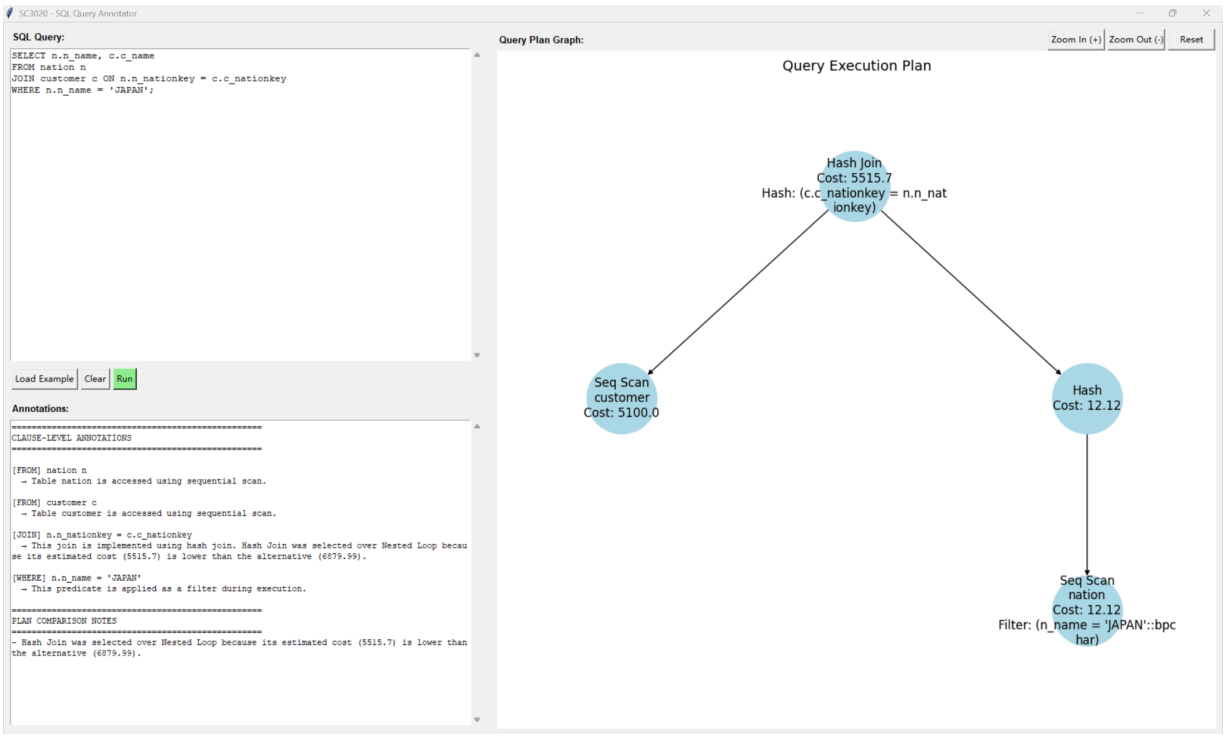


Between Condition



Hash Join

```
SELECT n.n_name, c.c_name
FROM nation n
JOIN customer c ON n.n_nationkey = c.c_nationkey
WHERE n.n_name = 'JAPAN';
```



LLM Usage Declaration

Component	LLM Used	Usage Description
interface.py (load_example)	Deepseek-V3.2	Creating pop-up window with list of example queries.
interface.py (panning & zooming)	Deepseek-V3.2	Implementing mouse drag to pan and scroll wheel to zoom